Fundamentals of Machine Learning

Tutorial 1, Extra Practice: Problems with Solutions

From learning problems to probabilistic classification. Same skills as Tutorial 1, asked differently: reverse-engineering, error-spotting, fitting from raw data, proofs, tool choice, evasion.

How to use this set

This set covers the same skills as Tutorial 1 but asks for them in different ways; the problems are not the tutorial with new numbers. The tutorial hands you costs, parameters and a protocol and asks you to compute. Here you work backwards from a decision rule to the costs behind it, find the mistakes in someone else's solution, fit the classifiers yourself from raw data rows, pick an operating point from a validation table, prove the facts everything rests on, choose the right tool for a situation, and derive when a detector can be evaded. Part B is twelve true-or-false statements that each need a one-line justification.

Each problem opens with a blue box holding the setup, then a yellow box listing the tools you need with a pointer [one-read §x] to the one-read document. Each sub-question is followed by its full solution. Difficulty: ⋆ direct application, ⋆⋆ several steps, ⋆⋆⋆ derivation or proof. All logarithms are natural.

Contents

1 Part A: Problems **1**

1.1 Problem 1: Reverse-engineer the decision rule . . . 1
1.2 Problem 2: Find the mistakes . . . 3
1.3 Problem 3: From raw rows to three classifiers . . . 4
1.4 Problem 4: Choosing the operating point from a validation table . . . 6
1.5 Problem 5: Five facts the tutorial relies on . . . 7
1.6 Problem 6: Which tool, which protocol . . . 9
1.7 Problem 7: Evading the detector . . . 10

2 Part B: True or false, with one line of justification **12**

1 Part A: Problems

1.1 Problem 1: Reverse-engineer the decision rule

Problem 1 (⋆⋆) 15 minutes

An intrusion detection system was deployed by a team that has since left. Its documentation says only: "raise an alert when the model's attack posterior is at least 0.125". Analysts estimate that handling a false alert costs about one hour of work; take that hour as the unit of cost.

Tools you need

Conditional risk $R(a \mid x) = \sum_y c(a, y)\, p(y \mid x)$ and the Bayes rule [one-read §2.2]. The threshold formula $t = C_{FP}/(C_{FP} + C_{FN})$ assumes correct decisions cost 0.

(a) Assuming correct decisions cost nothing, what cost did the team implicitly assign to a missed attack?

$t = C_{FP}/(C_{FP} + C_{FN})$ with $C_{FP} = 1$: $0.125 = 1/(1 + C_{FN}) \iff C_{FN} = 7$. The rule treats a missed attack as seven times as bad as a false alert.

(b) Correct decisions are not free: a true alert still costs analyst time. Take the general cost table below and derive the threshold on $p = p(\text{attack} \mid x)$. Then show that the table $a = 1$, $b = 2$, $M = 9$ gives the same threshold as (a), and explain why the two cost tables cannot be told apart from the rule alone.

	truth: benign	truth: attack
ignore	0	M
alert	a	b

ignore $R(\text{ignore} \mid x) = Mp,$

alert $R(\text{alert} \mid x) = a(1-p) + bp.$

Alert when $a(1-p) + bp < Mp \iff a < p\,(a + M - b) \iff p > \dfrac{a}{a + (M - b)}$.

With $a = 1$, $b = 2$, $M = 9$: $t = 1/(1 + 7) = 0.125$, the same as (a). Only the *differences* within each column matter: $a - 0$ is what a false alert costs beyond ignoring a benign flow, and $M - b = 7$ is what a miss costs beyond alerting on an attack. Adding a constant to a whole column ("every attack costs 2 whatever we do") changes nothing about the decision. So the rule determines the ratio of the two differences and nothing more; (a) was the special case $b = 0$.

(c) A third action, *block*, is added: it costs 5 on a benign flow (an outage) and 0.5 on an attack (the attack is stopped but still has to be investigated). With $a = 1$, $b = 2$, $M = 9$, derive the three decision regions.

$R(\text{block} \mid x) = 5(1-p) + 0.5p = 5 - 4.5p$, with $R(\text{ignore}) = 9p$ and $R(\text{alert}) = 1 + p$. Pairwise crossings:

$$\begin{aligned} \text{ignore vs alert:} &\quad 9p = 1 + p \iff p = 1/8 = 0.125, \\ \text{alert vs block:} &\quad 1 + p = 5 - 4.5p \iff p = 8/11 \approx 0.727, \\ \text{ignore vs block:} &\quad 9p = 5 - 4.5p \iff p = 10/27 \approx 0.370. \end{aligned}$$

Ignore is best below 0.125 (it beats alert there, and beats block since $0.125 < 0.370$). Alert is best between 0.125 and 0.727. Block is best above 0.727. The ignore/block crossing at 0.370 is irrelevant: alert beats both of them there (at $p = 0.37$: ignore 3.33, alert 1.37, block 3.33). With three actions, always check which pairwise crossings actually lie on the lower envelope.

(d) The analysts can handle at most 120 alerts per day. Of the 10,000 daily flows, 40 have posterior ≥ 0.5, another 80 have posterior in $[0.2, 0.5)$, and another 200 have posterior in $[0.125, 0.2)$. How many alerts does the cost-optimal rule of (a) produce? Propose a rule that respects the budget, and quantify what it gives up.

The rule "alert if $p \geq 0.125$" produces $40 + 80 + 200 = 320$ alerts per day, nearly three times the budget. The natural budget-respecting rule ranks flows by posterior and alerts on the top 120, which here means "alert if $p \geq 0.2$" ($40 + 80 = 120$). The ranking is the same the posterior already gives; only the cut-off moves.

What it gives up: the 200 flows in $[0.125, 0.2)$ are no longer examined. Their posteriors average about 0.16, so about $200 \times 0.16 \approx 32$ attacks per day go uninvestigated. Each of these alerts was

worth raising ($7 \times 0.16 - 1 \approx 0.12 > 0$ expected hours saved), so the budget is binding and costs the organisation about $32 \times 7 - 200 \approx 24$ hours per day in expectation. A budget is a constraint, not a cost: the correct response is to report that number, not to pretend the threshold 0.2 is cost-optimal.

1.2 Problem 2: Find the mistakes

Problem 2 ($\star\star$) **15 minutes**

A student was given the following exercise. *A logistic model was trained on data with attack prior* $\pi_t = 0.25$ *and is deployed where* $\pi_d = 0.05$*; assume prior shift and a calibrated training posterior. For one flow,* $r_t(x) = 0.8$*. Costs:* $C_{FP} = 2$*,* $C_{FN} = 6$*. On a test set of* 1,000 *flows with* 50 *attacks the model gives* $TP = 40$ *and* $FP = 60$*. Standardisation of the features was fitted on training and test data together. Decide what to do with the flow and assess the model.*
Their solution, numbered claim by claim:

(1) The cost threshold is $t = C_{FN}/(C_{FP} + C_{FN}) = 6/8 = 0.75$.
(2) Correcting for prior shift, $r_d(x) = r_t(x) \times \pi_d/\pi_t = 0.8 \times 0.2 = 0.16$.
(3) Since $0.16 < 0.75$, the flow is declared benign.
(4) Precision $= TP/(TP + FN) = 40/50 = 0.80$ and recall $= TP/(TP + FP) = 40/100 = 0.40$.
(5) $TN = 1000 - 50 - 60 = 890$, so accuracy $= (40 + 890)/1000 = 0.93$, which beats the all-benign baseline of 0.50; the model is useful.
(6) Fitting the standardisation on all the data is not leakage, because standardisation uses no labels.
(7) The threshold 0.75 is confirmed by the fact that it also gave the best F_1 on the test set.

Tools you need

Threshold [one-read §2.2]; prior-shift correction [one-read §4.7]; precision, recall, baselines, leakage, test-set reuse [one-read §3.5, 3.6].

(a) Go through the seven claims. For each, say whether it is right or wrong; if wrong, give the corrected statement or value.

(1) Wrong. The threshold is $C_{FP}/(C_{FP} + C_{FN}) = 2/8 = 0.25$. The student's formula gives the threshold for declaring *benign*, i.e. $1 - t$. Sanity check: a costly miss must *lower* the alert threshold.

(2) Wrong. Multiplying by π_d/π_t alone does not renormalise; it is not even guaranteed to stay below 1. The correct correction is

$$r_d = \frac{(\pi_d/\pi_t)\, r_t}{(\pi_d/\pi_t)\, r_t + \frac{1-\pi_d}{1-\pi_t}(1 - r_t)} = \frac{0.2 \times 0.8}{0.16 + \frac{0.95}{0.75} \times 0.2} = \frac{0.16}{0.16 + 0.2533} \approx 0.387.$$

(3) Wrong conclusion. With the corrected numbers, $0.387 \geq 0.25$: **alert**. The student's decision is wrong for two compounding reasons, not by a rounding error.

(4) Wrong, swapped. Precision $= TP/(TP + FP) = 40/100 = 0.40$; recall $= TP/(TP + FN) = 40/50 = 0.80$. Memory aid: precision is about the alerts raised (denominator contains FP); recall is about the attacks that exist (denominator contains FN).

(5) Half wrong. $TN = 890$ and accuracy 0.93 are right. The all-benign baseline is $950/1000 = 0.95$, not 0.50: the model's accuracy is *below* the baseline. The conclusion "useful" may still hold, but for a different reason: the baseline catches no attacks, while this model catches 80% of them at the price of 60 false alerts. Accuracy is the wrong metric to argue with here.

(6) Wrong. Leakage is any use of test information when building the model, not only label use. Means and standard deviations computed with the test rows let the test set shape the preprocessing. The effect is usually small, but the protocol is wrong: fit the scaler on training data only.

(7) Wrong. Choosing a threshold on the test set and then reporting test performance is selection on the test set; the reported F_1 is optimistically biased and the test set is no longer an independent check. Thresholds are chosen on validation data. Besides, a threshold should come from the costs (claim 1), not from F_1, which weighs false alerts and misses equally.

(b) Write the correct final answer in two sentences.

The flow has deployment posterior ≈ 0.39, above the cost threshold 0.25, so it should be alerted. On the test set the model has recall 0.80 and precision 0.40 (accuracy 0.93, below the 0.95 trivial baseline, which is irrelevant here); the figures are slightly optimistic because the scaler saw the test data, and the threshold should be re-chosen from the costs on a validation set.

1.3 Problem 3: From raw rows to three classifiers

Problem 3 ($\star\star\star$) **25 minutes**

Eight labelled hosts, two features each (x_1 = distinct internal hosts contacted per minute, x_2 = failed logins per minute, both already on comparable scales):

host	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
x_1	1	2	1	2	4	5	4	5
x_2	1	1	3	3	2	2	4	4
y	0	0	0	0	1	1	1	1

A new host has $x = (3, 4)$.

Tools you need

KNN and leave-one-out [one-read §2.4]; LDA with a shared covariance estimated by pooling, $\hat{\Sigma} = \frac{1}{2}(\hat{\Sigma}_0 + \hat{\Sigma}_1)$ with $\hat{\Sigma}_c = \frac{1}{n_c}\sum_{i \in c}(x_i - \hat{\mu}_c)(x_i - \hat{\mu}_c)^\top$ [one-read §4.4]; Gaussian Naive Bayes = one variance per feature per class [one-read §4.5].

(a) Classify x with 1-NN, 3-NN and 5-NN under Euclidean distance. Explain why 4-NN is ambiguous here.

Squared distances from $(3, 4)$: $h_7 = (4, 4)$: 1; $h_4 = (2, 3)$: 2; $h_8 = (5, 4)$: 4; $h_3 = (1, 3)$: 5; $h_5 = (4, 2)$: 5; $h_6 = (5, 2)$: 8; h_2: 10; h_1: 13.

- 1-NN: h_7, class **1**.
- 3-NN: h_7, h_4, h_8: two of class 1, one of class 0, so **1**.
- 5-NN: h_7, h_4, h_8, h_3, h_5: three of class 1, so **1**.

4-NN is ambiguous because the fourth and fifth neighbours, h_3 (class 0) and h_5 (class 1), are tied at distance $\sqrt{5}$. Depending on which is admitted, the vote is 2–2 (a tie) or 3–1. Ties in distance and ties in the vote both need a rule fixed in advance (odd k, distance weighting, or a deterministic tie-break).

(b) Compute the leave-one-out error of 1-NN on the eight training hosts. Does the result mean 1-NN generalises well here?

For each host, the nearest *other* host: $h_1 \leftrightarrow h_2$ (distance 1), $h_3 \leftrightarrow h_4$ (1), $h_5 \leftrightarrow h_6$ (1), $h_7 \leftrightarrow h_8$ (1). Each pair shares a label, so every held-out host is classified correctly: LOO error $= 0/8 = 0$.

It means little. The two classes form well-separated clusters, so any sensible method scores 0 on these eight points. Leave-one-out tells you how the method behaves on points that look like the training set; it says nothing about hosts between the clusters, such as the query $(3, 4)$, where the methods below will disagree once the prior changes.

(c) Fit LDA: compute the class means, the pooled covariance, and the boundary with equal priors. Classify x and give its posterior.

Means: $\hat{\mu}_0 = (1.5, 2)$, $\hat{\mu}_1 = (4.5, 3)$.

Within class 0 the deviations are $(\pm 0.5, \pm 1)$ in all four sign combinations, so the variances are 0.25 and 1, and the cross-products $(\pm 0.5)(\pm 1)$ cancel: $\hat{\Sigma}_0 = \mathrm{diag}(0.25, 1)$. Class 1 has exactly the same pattern of deviations, so $\hat{\Sigma}_1 = \hat{\Sigma}_0$ and the pooled $\hat{\Sigma} = \mathrm{diag}(0.25, 1)$, with $\hat{\Sigma}^{-1} = \mathrm{diag}(4, 1)$.

LDA scores $s_c(x) = x^\top \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \ln \pi_c$:

$$\hat{\Sigma}^{-1}\hat{\mu}_0 = (6, 2), \quad \tfrac{1}{2}\hat{\mu}_0^\top \hat{\Sigma}^{-1}\hat{\mu}_0 = \tfrac{1}{2}(9 + 4) = 6.5 \;\Rightarrow\; s_0(x) = 6x_1 + 2x_2 - 6.5 + \ln \pi_0,$$
$$\hat{\Sigma}^{-1}\hat{\mu}_1 = (18, 3), \quad \tfrac{1}{2}\hat{\mu}_1^\top \hat{\Sigma}^{-1}\hat{\mu}_1 = \tfrac{1}{2}(81 + 9) = 45 \;\Rightarrow\; s_1(x) = 18x_1 + 3x_2 - 45 + \ln \pi_1.$$

With equal priors, $s_1 - s_0 = 12x_1 + x_2 - 38.5$, so the boundary is $12x_1 + x_2 = 38.5$. It passes through the midpoint $(3, 2.5)$ of the means, as it must with equal priors.

At $x = (3, 4)$: $s_1 - s_0 = 36 + 4 - 38.5 = 1.5$, so $P(y = 1 \mid x) = \sigma(1.5) \approx 0.82$: class **1**.

(d) The boundary's normal vector is $(12, 1)$: almost vertical, although the means differ in both coordinates. Explain, and contrast with the Euclidean KNN of (a).

The normal is $\hat{\Sigma}^{-1}(\hat{\mu}_1 - \hat{\mu}_0) = (4 \times 3,\ 1 \times 1)$. The mean difference $(3, 1)$ is weighted by the inverse variances: x_1 has variance 0.25, so a difference of 3 in x_1 is 6 standard deviations, while the difference of 1 in x_2 is 1 standard deviation. LDA trusts x_1 twelve times more. Euclidean KNN weights both coordinates equally, which is why in (a) $h_4 = (2, 3)$ counted as a close neighbour: it is near in raw distance but, by LDA's measure, far on the informative axis. KNN inherits the geometry you give it; LDA estimates one from the data.

(e) Now fit Gaussian Naive Bayes (one variance per feature and per class). Show that it gives exactly the same boundary as LDA on this data, and state what would have to change for the two to differ.

Gaussian NB models $p(x \mid c) = \prod_j \mathcal{N}(x_j;\ \mu_{cj}, \sigma_{cj}^2)$, i.e. a Gaussian with diagonal covariance per class. Here the per-class variances are $(0.25, 1)$ for *both* classes, so NB uses $\Sigma_0 = \Sigma_1 = \mathrm{diag}(0.25, 1)$, which is the LDA pooled covariance, and the same means. Same model, same boundary.

They differ when (i) the per-class variances differ, in which case NB's boundary gains quadratic terms (it is a diagonal QDA), or (ii) the features are correlated within a class, in which case LDA estimates the off-diagonal terms and NB ignores them. On this data the within-class covariance is exactly 0, so (ii) does not arise.

(f) Attacks are actually 10% of hosts. Redo the LDA classification of x with $\pi_1 = 0.1$. Then say which of the three methods you would trust least with eight training points, and why.

$s_1 - s_0 = 12x_1 + x_2 - 38.5 + \ln(0.1/0.9) = 1.5 - 2.197 = -0.697$, so $P(y = 1 \mid x) = \sigma(-0.697) \approx 0.33$: class **0**. The boundary moved to $12x_1 + x_2 = 40.70$, away from the rare class, and the query flipped.

With eight points, trust least the method with the most parameters per data point. LDA estimates $2 + 2 + 3 = 7$ numbers (two means and a symmetric 2×2 covariance) from 8 rows; Gaussian NB estimates $2 + 2 + 4 = 8$ (it drops the covariance but doubles the variances). Both are stretched. 1-NN estimates nothing, but it is the highest-variance predictor of all: its answer on the query depends on a single training host. None is trustworthy here; the honest answer is that 8 points cannot adjudicate between them, and the posterior 0.82 from (c) is a model statement, not a measurement.

1.4 Problem 4: Choosing the operating point from a validation table

Problem 4 (⋆⋆) **20 minutes**

A scoring model was run on a validation set of 550 flows, 50 of them attacks. The scores are grouped into bins:

score bin	$[0.9, 1]$	$[0.7, 0.9)$	$[0.5, 0.7)$	$[0.3, 0.5)$	$[0, 0.3)$
attacks	20	15	10	4	1
benign	2	8	20	70	400

Costs: $C_{FP} = 1$, $C_{FN} = 10$.

Tools you need

Precision, recall, F_1, false-positive rate, ROC [one-read §3.6]; the cost threshold [one-read §2.2]; calibration [one-read §4.7].

(a) For each threshold $t \in \{0.9, 0.7, 0.5, 0.3, 0\}$ (alert if score $\geq t$), compute TP, FP, FN, precision, recall, F_1 and the false-positive rate.

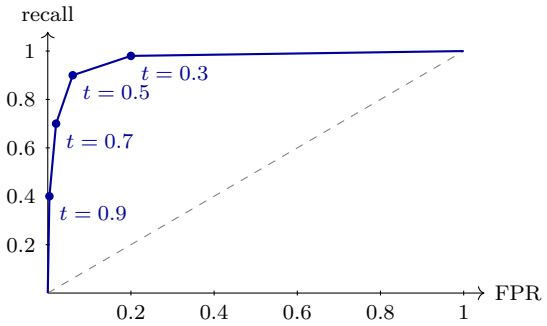
Accumulate the bins from the top:

t	TP	FP	FN	precision	recall	F_1	FPR
0.9	20	2	30	0.909	0.40	0.556	0.004
0.7	35	10	15	0.778	0.70	0.737	0.020
0.5	45	30	5	0.600	0.90	0.720	0.060
0.3	49	100	1	0.329	0.98	0.492	0.200
0	50	500	0	0.091	1.00	0.167	1.000

Recall rises and FPR rises as t falls; precision falls here, though in general it need not be monotone.

(b) Sketch the ROC points. Which threshold maximises F_1, and which minimises the expected cost $C_{FP} \cdot FP + C_{FN} \cdot FN$? Why do they differ?

ROC points (FPR, recall): $(0.004, 0.40)$, $(0.02, 0.70)$, $(0.06, 0.90)$, $(0.20, 0.98)$, $(1, 1)$, plus $(0, 0)$ for "never alert".



Costs: $t = 0.9$: $2 + 300 = 302$; $t = 0.7$: $10 + 150 = 160$; $t = 0.5$: $30 + 50 = \mathbf{80}$; $t = 0.3$: $100 + 10 = 110$; $t = 0$: 500.

F_1 is maximised at $t = 0.7$ (0.737), expected cost is minimised at $t = 0.5$ (80). They differ because F_1 weighs a false alert and a miss equally (it is a function of precision and recall only), while the cost says a miss is ten times worse, which pushes the threshold down. Use the metric that encodes the real costs; F_1 is a convention, not a decision rule.

(c) The Bayes threshold for these costs is $1/11 \approx 0.09$, yet the best threshold on the scores was 0.5. Compute, for each bin, the empirical fraction of attacks and compare it with the bin's scores. Resolve the apparent contradiction.

bin	$[0.9, 1]$	$[0.7, 0.9)$	$[0.5, 0.7)$	$[0.3, 0.5)$	$[0, 0.3)$
attack fraction	$20/22 = 0.91$	$15/23 = 0.65$	$10/30 = 0.33$	$4/74 = 0.054$	$1/401 = 0.002$

The scores are *not calibrated posteriors*: a flow scored around 0.6 is an attack only a third of the time, and one scored around 0.4 only 5% of the time. The model is over-confident in the middle range. The Bayes rule "alert if the true posterior exceeds $1/11$" applied to the *empirical* fractions says: alert on the bins with fraction > 0.09, i.e. the top three bins, which is exactly "score ≥ 0.5". No contradiction: 0.5 on the score scale corresponds to roughly 0.09 on the posterior scale. The threshold formula is only meaningful on calibrated probabilities; on raw scores, choose the threshold empirically on validation data, or calibrate the scores first.

(d) Use the empirical fractions to build a calibration map (bin $\to$ fraction) and apply it to a new flow with score 0.55. With calibrated outputs, what threshold would you now use, and does the decision for this flow change?

Score 0.55 falls in $[0.5, 0.7)$, whose calibrated posterior is 0.33. With calibrated outputs the Bayes threshold $1/11 \approx 0.09$ applies directly, and $0.33 > 0.09$: alert. The decision is the same as with the empirical threshold 0.5 on raw scores, as it must be, since the calibration map is monotone and both rules alert on the same three bins. What calibration adds is a correct *probability* to report and the ability to reuse the cost formula when the costs change, without re-sweeping thresholds.

(e) At deployment attacks are 1% of flows rather than $50/550 \approx 9\%$. Which quantities in the table of (a) stay valid and which do not, assuming only the prior changes?

Recall and FPR are computed within a class (attacks only, benign only), so they are unchanged by the prior. Precision, accuracy and F_1 mix the two classes and will change: at 1% prevalence with recall 0.9 and FPR 0.06, precision becomes $\frac{0.9\times0.01}{0.9\times0.01+0.06\times0.99} \approx 0.13$ instead of 0.60. The calibration map of (c) also becomes wrong (every bin's attack fraction drops), and must be corrected by the prior-shift formula [one-read §4.7]. The ROC curve is the one object in this problem that survives the prior change.

1.5 Problem 5: Five facts the tutorial relies on

Problem 5 ($\star\star\star$) — 25 minutes

Prove the following, in the notation of the lectures. X is the input, $Y \in \{0, 1\}$ the label, $p(x) = P(Y = 1 \mid X = x)$, and $\ell(y, a)$ a loss for action a.

Tools you need

Tower property $\mathbb{E}[Z] = \mathbb{E}[\mathbb{E}[Z \mid X]]$; conditional risk [one-read §2.2]; empirical and population risk [one-read §3.1]; the logistic gradient [one-read §4.2].

(a) The Bayes rule is optimal. Let $a^\star(x) \in \arg\min_a \sum_y \ell(y, a)\, P(Y = y \mid X = x)$. Show that for every decision rule $a(\cdot)$, $\mathbb{E}[\ell(Y, a(X))] \geq \mathbb{E}[\ell(Y, a^\star(X))]$.

By the tower property, $\mathbb{E}[\ell(Y, a(X))] = \mathbb{E}\big[\,\mathbb{E}[\ell(Y, a(X)) \mid X]\,\big]$. Conditioning on $X = x$, the inner expectation is the conditional risk of the action $a(x)$:

$$\mathbb{E}[\ell(Y, a(X)) \mid X = x] = \sum_y \ell(y, a(x))\, P(Y = y \mid X = x) = R(a(x) \mid x).$$

By definition of $a^\star$, $R(a(x) \mid x) \geq R(a^\star(x) \mid x)$ for every x. Taking the expectation over X of both sides of a pointwise inequality preserves it:

$$\mathbb{E}[\ell(Y, a(X))] = \mathbb{E}[R(a(X) \mid X)] \geq \mathbb{E}[R(a^\star(X) \mid X)] = \mathbb{E}[\ell(Y, a^\star(X))]. \qquad \blacksquare$$

The argument is purely pointwise: minimising the expected loss over *functions* reduces to minimising a number at each x. This is why every threshold rule in the tutorial is derived one x at a time.

(b) The Bayes error. For the 0–1 loss, show that the minimal achievable error is $R^\star = \mathbb{E}[\min\{p(X),\ 1 - p(X)\}]$, that $R^\star \leq \frac{1}{2}$, and that $R^\star = 0$ if and only if $p(X) \in \{0, 1\}$ with probability 1.

With $\ell(y, a) = \mathbf{1}(y \neq a)$, the conditional risks are $R(1 \mid x) = 1 - p(x)$ and $R(0 \mid x) = p(x)$, so the Bayes rule predicts the more probable class and its conditional risk is $\min\{p(x), 1 - p(x)\}$. By (a), $R^\star = \mathbb{E}[\min\{p(X), 1 - p(X)\}]$.

Since $\min\{p, 1 - p\} \leq \frac{1}{2}$ for every $p \in [0, 1]$, $R^\star \leq \frac{1}{2}$ (equality when $p(X) = \frac{1}{2}$ almost surely, i.e. X carries no information).

$\min\{p, 1 - p\} \geq 0$ with equality iff $p \in \{0, 1\}$. A non-negative random variable has expectation 0 iff it is 0 with probability 1. Hence $R^\star = 0$ iff $p(X) \in \{0, 1\}$ almost surely: the label is a deterministic function of the features. $\blacksquare$

This is Problem 1(b) of the tutorial in one line: two flows with the same x and different labels force $0 < p(x) < 1$ there, hence $R^\star > 0$ whenever such x have positive probability.

(c) Precision depends on the prevalence. Let $\pi = P(Y = 1)$, $\rho = P(\text{alert} \mid Y = 1)$ (recall) and $\phi = P(\text{alert} \mid Y = 0)$ (false-positive rate). Show that $\text{precision} = \dfrac{\rho\pi}{\rho\pi + \phi(1 - \pi)}$ and that it is increasing in π when $\rho, \phi > 0$.

Precision is $P(Y = 1 \mid \text{alert})$. By Bayes' rule,

$$P(Y = 1 \mid \text{alert}) = \frac{P(\text{alert} \mid Y = 1)P(Y = 1)}{P(\text{alert} \mid Y = 1)P(Y = 1) + P(\text{alert} \mid Y = 0)P(Y = 0)} = \frac{\rho\pi}{\rho\pi + \phi(1 - \pi)}.$$

Write it as $1/(1 + \frac{\phi}{\rho} \cdot \frac{1-\pi}{\pi})$. The map $\pi \mapsto (1 - \pi)/\pi$ is decreasing on $(0, 1]$, so $\frac{\phi}{\rho} \cdot \frac{1-\pi}{\pi}$ is decreasing and the precision is increasing in π. $\blacksquare$

Recall and FPR are properties of the classifier; precision is a property of the classifier *and* the population it is used on. That is Problem 4(e) in general.

(d) Training error is optimistic. Let $\widehat{R}_n(\theta) = \frac{1}{n}\sum_i \ell(y_i, f_\theta(x_i))$ on an i.i.d. sample, $R(\theta) = \mathbb{E}[\ell(Y, f_\theta(X))]$, $\hat{\theta} \in \arg\min_\theta \widehat{R}_n(\theta)$ and $\theta^\star \in \arg\min_\theta R(\theta)$. Show that $\mathbb{E}[\widehat{R}_n(\hat{\theta})] \leq R(\theta^\star) \leq \mathbb{E}[R(\hat{\theta})]$.

Left inequality. $\hat{\theta}$ minimises $\widehat{R}_n$, so for the fixed (non-random) parameter $\theta^\star$, $\widehat{R}_n(\hat{\theta}) \leq \widehat{R}_n(\theta^\star)$ on every sample. Take expectations: $\mathbb{E}[\widehat{R}_n(\hat{\theta})] \leq \mathbb{E}[\widehat{R}_n(\theta^\star)]$. Since $\theta^\star$ does not depend on the sample, $\widehat{R}_n(\theta^\star)$ is an average of i.i.d. copies of $\ell(Y, f_{\theta^\star}(X))$, so $\mathbb{E}[\widehat{R}_n(\theta^\star)] = R(\theta^\star)$.

Right inequality. $\theta^\star$ minimises R, so $R(\theta^\star) \leq R(\theta)$ for every θ, in particular for $\theta = \hat{\theta}$ on every sample; take expectations. $\blacksquare$

Read it as: training error of the fitted model $\leq$ best possible population error $\leq$ population error of the fitted model. The expected gap $\mathbb{E}[R(\hat{\theta})] - \mathbb{E}[\widehat{R}_n(\hat{\theta})]$ is the *optimism* of the training error, and it is why a validation set is needed at all. Note where the proof used that $\theta^\star$ is fixed: the same step fails for $\hat{\theta}$, because $\hat{\theta}$ was chosen by looking at the sample.

(e) Logistic regression is calibrated on average. For $r_i = \sigma(\theta^\top x_i + \theta_0)$ trained by minimising the summed cross-entropy, show that at the optimum $\sum_i r_i = \sum_i y_i$, i.e. the predicted number of attacks equals the observed number. Why does this fail if the intercept θ_0 is omitted?

Write $s_i = \theta^\top x_i + \theta_0$. From Tutorial 1, $\partial \ell_i / \partial s_i = r_i - y_i$, and $\partial s_i / \partial \theta_0 = 1$. The derivative of the total loss with respect to the intercept is therefore

$$\frac{\partial}{\partial \theta_0} \sum_i \ell_i = \sum_i (r_i - y_i),$$

which must vanish at a minimum: $\sum_i r_i = \sum_i y_i$. ■

Without an intercept there is no parameter whose gradient is $\sum_i (r_i - y_i)$; the optimality conditions are $\sum_i (r_i - y_i) x_{ij} = 0$ for each feature j, which constrain other weighted sums but not the plain one. This is why the balanced-sample trick in Problem 2 is repaired by changing the intercept alone [one-read §4.7]: the intercept is the parameter that sets the average predicted rate.

1.6 Problem 6: Which tool, which protocol

Problem 6 (⋆⋆) 15 minutes

For each situation choose one of: k-nearest neighbours, logistic regression, LDA, (Bernoulli or Gaussian) Naive Bayes, or a window/sequence detector feeding a second classifier. Give one reason for the choice, one risk to watch, and the right validation split.

(i) 300 labelled phishing emails and 2,000 legitimate ones; features are 5,000 binary word indicators.

(ii) Two million flows, 40 numeric and strongly correlated features; the output must be a probability that will be thresholded by costs that change per customer.

(iii) Two well-chosen features, abundant data, and a scatter plot showing attacks as a small island inside the benign region.

(iv) Several features are frequently unavailable at decision time (a sensor is off); the model must still answer.

(v) The team must flag flows that look like *nothing* seen in training, not just flows that look like known attacks.

(vi) Each event looks innocuous; the attack is visible only as a pattern across many events from one source.

Tools you need

Local versus parametric [one-read §2.4, 2.5]; discriminative versus generative and their trade-offs [one-read §4.6]; handling missing and unfamiliar inputs [one-read §4.8]; splits and leakage [one-read §3.6].

(a) Give your six answers.

(i) **Bernoulli Naive Bayes.** Reason: 5,000 parameters per class estimated from counts, which is feasible with 300 positives, where logistic regression with 5,000 weights would overfit badly without strong regularisation. Risk: correlated words (“verify” and “account”) double-count evidence, so posteriors are over-confident; calibrate before thresholding. Split: by campaign (group), so near-duplicate emails of one campaign are not on both sides.

(ii) **Logistic regression.** Reason: it models $p(y \mid x)$ directly, shares weight among correlated features instead of multiplying their evidence, and with two million rows its outputs are usually well calibrated, which is what cost thresholds need. Risk: a linear boundary may underfit; check with validation and add features or interactions. Split: by time (train on earlier, validate and test on later), since flow traffic drifts.

(iii) k-**NN.** Reason: an island inside another class is a non-linear boundary that a local method captures without any parametric assumption, and with two features and plenty of data the neighbourhoods are dense. Risk: scaling of the two features decides the answer; choose it on validation. Split: random or grouped by source host, and never standardise before splitting.

(iv) **A generative model (Gaussian Naive Bayes or LDA).** Reason: with an explicit $p(x \mid c)$ a missing feature can be integrated out (for Naive Bayes, simply drop its factor), whereas a discriminative model needs the whole vector. Risk: the independence or Gaussian assumptions may be wrong; check calibration on validation. Split: stratified by whether the sensor was off, so the missing-data regime is represented on both sides.

(v) **A generative model with a density check.** Reason: a generative classifier gives $p(x) = \sum_c \pi_c\, p(x \mid c)$, so a flow with very low $p(x)$ can be flagged as unfamiliar; a discriminative model only says which side of its boundary the flow is on, with confidence that can be high far from all data. Risk: the density is only as good as the model's assumptions in the tails. Split: hold out whole attack families to test whether unseen families are flagged.

(vi) **Window features and a second classifier.** Reason: the signal is in counts, distinct values and score sums over recent events from the same source, which no per-event model can see. Risk: future leakage when building windows, and the choice of grouping key behind NAT. Split: by episode and by time, with all windows of an episode on the same side.

(b) Situation (ii) again, but the customer now also wants to know "which of the 40 features made this flow suspicious". Does the choice change?

No, and this is a point in favour of logistic regression: its score is $\theta_0 + \sum_j \theta_j x_j$, so the contribution of each feature to this flow's score is the term $\theta_j x_j$, which can be listed and ranked. The caveat is the correlation: with strongly correlated features the weights share credit in an arbitrary way, so "feature 7 contributed +1.3" describes the model, not a causal fact about the attack. A Naive Bayes model would give an equally readable per-feature decomposition (the log likelihood ratios), but with the double-counting problem.

1.7 Problem 7: Evading the detector

Problem 7 $(\star\star\star)$ **20 minutes**

Two detectors watch failed login attempts from one source. Each attempt carries a score s from a per-attempt classifier.

- **Window detector:** alert when at least $k = 4$ attempts fall in the half-open window $(t - W,\ t]$ ending at the current attempt, with $W = 30$ s.
- **Decaying accumulator:** $A(t_i) = A(t_{i-1})\, e^{-(t_i - t_{i-1})/\tau} + s_i$, $A = 0$ initially, alert when $A \geq \theta$; here $\tau = 30$ s and $\theta = 0.55$.

A patient attacker sends one attempt every Δ seconds, each with score $s = 0.2$.

Tools you need

Window features and accumulation [one-read §4.8]; geometric series $\sum_{i=0}^{n-1} q^i = (1 - q^n)/(1 - q)$.

(a) Show that the window detector alerts if and only if $\Delta < W/(k-1)$, and compute the bound.

At the attempt at time t, the window contains the attempts at $t, t - \Delta, \ldots, t - j\Delta$ for every j with $t - j\Delta > t - W$, i.e. $j\Delta < W$. It holds k attempts iff $j = k - 1$ qualifies, i.e. $(k-1)\Delta < W \iff \Delta < W/(k-1)$. With $W = 30$ and $k = 4$: $\Delta < 10$ s. An attacker spacing attempts 10 s apart or more is never detected, however long the campaign lasts, since every window then holds at most 3 attempts.

(b) For the accumulator with periodic attempts, write $q = e^{-\Delta/\tau}$. Show that $A_n = s\,\dfrac{1-q^n}{1-q}$ after n attempts, that A_n increases towards $A^\star = s/(1-q)$, and that the detector eventually alerts iff $A^\star > \theta$, i.e. iff $\Delta < \tau \ln \dfrac{1}{1-s/\theta}$ (assume $s < \theta$).

Between consecutive attempts the gap is Δ, so the recursion is $A_n = qA_{n-1} + s$ with $A_0 = 0$. Unrolling, $A_n = s(1 + q + \cdots + q^{n-1}) = s\,\frac{1-q^n}{1-q}$. Since $0 < q < 1$, q^n decreases to 0, so A_n increases strictly towards $A^\star = s/(1-q)$ without ever reaching it.

Hence $A_n \geq \theta$ for some n iff $A^\star > \theta$ (if $A^\star > \theta$ the increasing sequence crosses θ at some finite n; if $A^\star \leq \theta$ it never does, since $A_n < A^\star$). Now

$$\frac{s}{1-q} > \theta \iff 1-q < \frac{s}{\theta} \iff q > 1 - \frac{s}{\theta} \iff -\frac{\Delta}{\tau} > \ln\left(1 - \frac{s}{\theta}\right) \iff \Delta < \tau \ln \frac{1}{1-s/\theta}. \qquad \blacksquare$$

(c) Compute the bound in (b) for $s = 0.2$, $\theta = 0.55$, $\tau = 30$. Which attackers does the accumulator catch that the window detector misses?

$1 - s/\theta = 1 - 0.3636 = 0.6364$, $\ln(1/0.6364) = 0.452$, so $\Delta < 30 \times 0.452 \approx 13.6$ s. The window detector needs $\Delta < 10$ s. Attackers spacing attempts between 10 and 13.5 s are caught by the accumulator and missed by the window detector. Check: at $\Delta = 13$, $A^\star = 0.2/(1 - e^{-13/30}) = 0.569 > 0.55$; at $\Delta = 14$, $A^\star = 0.536 < 0.55$.

(d) For $\Delta = 5$ s, how many attempts does the accumulator need before alerting, and at what time? Compare with the window detector.

$q = e^{-5/30} = 0.8465$, $A^\star = 0.2/0.1535 = 1.303$. The values are $A_1 = 0.200$, $A_2 = 0.369$, $A_3 = 0.513$, $A_4 = 0.634$: the first value ≥ 0.55 is A_4, at the fourth attempt, 15 s after the first. The window detector also alerts at the fourth attempt (four attempts within 15 s < 30 s), at the same time. For fast attackers the two agree; they differ on the slow ones.

(e) A legitimate but clumsy host produces a failed attempt every 5 s with score 0.05; a legitimate automation script with a stale password produces one every 5 s with score 0.10. Does the accumulator raise a false alert on either? What does this say about the choice of τ?

With $\Delta = 5$, $1 - q = 0.1535$. Clumsy host: $A^\star = 0.05/0.1535 = 0.326 < 0.55$, no alert ever. Script: $A^\star = 0.10/0.1535 = 0.651 > 0.55$, so it *will* alert after enough attempts (here $A_n \geq 0.55$ needs $1 - q^n \geq 0.844$, i.e. $n \geq 12$, one minute in).

The parameters τ and θ trade two frontiers against each other: a larger τ (slower forgetting) catches slower attackers, but it also lets low-score legitimate activity pile up into false alerts. Both frontiers are governed by the same quantity $s/(1-e^{-\Delta/\tau})$, so one cannot be moved without the other. The only principled way to set (τ, θ) is on validation data containing both slow attacks and noisy legitimate hosts.

(f) The attacker learns the detector's parameters. What is the cheapest change to their behaviour, and what does that imply for how such thresholds should be managed?

Slow down to $\Delta \geq 14$ s: the accumulator saturates below θ, the window never fills, and the per-attempt classifier never fires (each attempt scores 0.2). The campaign takes 40% longer and is invisible. Any fixed, known threshold on a rate is a published speed limit. Implications: the parameters should be treated as secrets and rotated, randomised, or learned (a second classifier on window features generalises beyond a hand-set rule), and the per-attempt score itself should include features that a slowed attacker cannot change, such as the spread of usernames tried.

2 Part B: True or false, with one line of justification

Decide each statement and justify it in one line. A bare "true" or "false" earns nothing.

(1) If two flows have identical features and different labels, no amount of additional training data can bring the error on such flows to zero.

True. For that x, $0 < p(x) < 1$, so the Bayes error $\min\{p, 1-p\}$ is positive; more data can only approach the Bayes error, not beat it (Problem 5(b)).

(2) Lowering the alert threshold never decreases recall and never increases precision.

False. Recall indeed cannot decrease (more flows are alerted). Precision can go *either* way: if the newly alerted flows are mostly attacks, precision rises. It is non-increasing only in special cases such as Problem 4.

(3) In LDA with a shared covariance, changing the class priors rotates the decision boundary.

False. The prior enters only through the constant $\ln(\pi_1/\pi_0)$, so the boundary translates parallel to itself; its normal $\Sigma^{-1}(\mu_1 - \mu_0)$ is unchanged.

(4) With Laplace smoothing, Naive Bayes can still output a posterior of exactly 0 for some class.

False (with positive priors). Every smoothed likelihood is at least $1/(n_c + 2) > 0$, so every class score is a product of positive factors.

(5) Of two logistic models fitted to the same data, the one with the lower training cross-entropy also has the lower training error rate.

False. Cross-entropy rewards confidence on correct examples; a model can lower it while flipping a borderline example to the wrong side. Only the minimised criterion is guaranteed to improve.

(6) For the 0–1 loss, the Bayes error is at most 1/2.

True. It equals $\mathbb{E}[\min\{p(X), 1-p(X)\}]$ and the minimum of p and $1-p$ never exceeds 1/2.

(7) If the best of 100 configurations is chosen by validation error, that configuration's validation error is an unbiased estimate of its population error.

False. The minimum of 100 noisy estimates is biased downward; the winner was partly selected for its luck. Score it again on untouched data.

(8) For a logistic model, the rule "alert if $r(x) \geq t$" is the same as "alert if $s(x) \geq \ln \frac{t}{1-t}$".

True. σ is strictly increasing with inverse the logit, so thresholds transfer exactly between the two scales.

(9) The prior-shift correction can change which of two flows has the higher posterior.

False. The correction adds a constant to the log-odds of every flow, which is a strictly increasing transform of the posterior; rankings, and hence the ROC curve, are unchanged.

(10) Standardising the test features using the mean and standard deviation computed on the training set is leakage.

False. That is the correct procedure: the statistics come from training data only and are merely *applied* to the test set. Leakage would be computing them on the test rows.

(11) A 1-NN classifier always has zero error on its own training set.

True, provided no two training points share the same features with different labels. Each training point is its own nearest neighbour at distance 0. With duplicated features and conflicting labels, ties make the claim fail.

> **(12)** A detector that alerts on four failed attempts in 30 seconds can be replicated by a per-attempt classifier with a suitably chosen threshold.

False. A per-attempt classifier sees one attempt's features; the count of recent attempts from the same source is not among them unless it is added as a window feature, which is precisely the second detector.